Lab 3: Dynamic Configuration Management with Hydra

- **Course:** Engineering of Intelligent Models
- **Module:** M1 – MLOps Foundations
- **Focus Tool:** Hydra (by Meta)
- **Format:** Hands-on Laboratory
- **Branch:** `Lab3`

1. Goal of the Lab

Currently, our `get_data.py` script has hardcoded API coordinates (latitude, longitude) and date ranges. If we want to fetch data for a different city, we must manually edit the Python code. This breaks reproducibility and makes automation impossible.

In this lab, you will learn how to:

1. **Initialise Hydra** to replace hardcoded variables with hierarchical YAML configuration files.
2. **Implement Config Groups** to modularise settings (e.g., separating API configurations from file path configurations).
3. **Use Command Line Overrides** to alter script behaviour dynamically without touching a single line of Python.

> **NOTE:** While we are applying Hydra to our data ingestion today, this same framework will be crucial in later labs. When we implement our forecasting models (**LSTM, GRU, and Prophet**), Hydra will allow us to define model architectures, learning rates, and perform automated **Hyperparameter Optimization (Sweeps)** with a single command.

2. Version Control: Switching Branches

Ensure your Lab 2 work (DVC integration) is safely committed before starting this new feature.

After commiting and pushing on branch `Lab2`, switch to a new branch for Lab 3.

In [ ]:
```
# Create and switch to the Lab3 branch
!git checkout -b Lab3
```

3. Structuring the `conf/` Directory

Hydra thrives on composition. Instead of one massive configuration file, we build a tree of smaller, logical YAML files.

Update your `requirements.txt` and `pip install` them to ensure you have the correct versions for this lab:

```
mlflow==3.10.0
pandas==2.3.3
dvc==3.66.1
openmeteo-requests==1.7.5
requests-cache==1.3.0
retry-requests==2.0.0
pyyaml==6.0.3
hydra-core==1.3.2
omegaconf==2.3.0
```

Now, populate the `conf/` directory by creating the following files:

1. `conf/paths/default.yaml`: Defines where we save data

```yaml
# Configuration for file paths
raw_data_dir: "data/raw" # Directory to store raw data fetched from the API
processed_data_dir: "data/processed" # Directory to store processed data ready for modeling (used in later labs)
```

2. `conf/api/openmeteo.yaml`: Contains API parameters

```yaml
# Configuration for Open-Meteo API parameters
params:
  latitude: 38.801 # Porto, Portugal
  longitude: -9.3783
  start_date: "2026-02-06"
  end_date: "2026-02-20"
  hourly:
    - "temperature_2m"
    - "relative_humidity_2m"
    - "precipitation"
  timezone: "auto"
```

3. `conf/config.yaml`: The main configuration file that composes the above

```yaml
# This tells Hydra to load the default configurations from our sub-folders
defaults:
  - paths: default
  - api: openmeteo
  - _self_

project_name: "EMI-WeatherForecast"
```

4. Refactoring Data Ingestion with Hydra

Now, we must rewrite `src/ingestion/get_data.py` to accept these configurations dynamically via Hydra's `@hydra.main` decorator.

Replace the contents of `src/ingestion/get_data.py` with the following:

```python
import openmeteo_requests
import pandas as pd
import requests_cache
from retry_requests import retry
from datetime import datetime
import os
import hydra
from omegaconf import DictConfig, OmegaConf


# The decorator tells Hydra where to find the config files
# config_path is relative to the location of this python script
@hydra.main(version_base=None, config_path="../../conf", config_name="config")
def fetch_weather_data(cfg: DictConfig):
    print(f"Starting ingestion for project: {cfg.project_name}")

    cache_session = requests_cache.CachedSession('.cache', expire_after=-1)
    retry_session = retry(cache_session, retries=5, backoff_factor=0.2)
    openmeteo = openmeteo_requests.Client(session=retry_session)

    url = "https://archive-api.open-meteo.com/v1/archive"
```

```
    # Convert the Hydra DictConfig into a standard Python dictionary for the API
    params = OmegaConf.to_container(cfg.api.params, resolve=True)

    print(f"Requesting data for coordinates: {params['latitude']}N,
{params['longitude']}E")
    responses = openmeteo.weather_api(url, params=params)
    response = responses[0]
    hourly = response.Hourly()

    hourly_data = {
        "date": pd.date_range(
            start=pd.to_datetime(hourly.Time() + response.UtcOffsetSeconds(),
unit="s", utc=True),
            end=pd.to_datetime(hourly.TimeEnd() + response.UtcOffsetSeconds(),
unit="s", utc=True),
            freq=pd.Timedelta(seconds=hourly.Interval()),
            inclusive="left"
        ),
        "temperature_2m": hourly.Variables(0).ValuesAsNumpy(),
        "relative_humidity_2m": hourly.Variables(1).ValuesAsNumpy(),
        "precipitation": hourly.Variables(2).ValuesAsNumpy()
    }

    df = pd.DataFrame(data=hourly_data)

    # Use the path defined in conf/paths/default.yaml
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"weather_{timestamp}.csv"

    # Ensure the directory exists
    os.makedirs(cfg.paths.raw_data_dir, exist_ok=True)
    output_path = os.path.join(cfg.paths.raw_data_dir, filename)

    df.to_csv(output_path, index=False)
    print(f"Data successfully ingested: {output_path}")


if __name__ == "__main__":
    fetch_weather_data()
```

Code Explanation:

- `@hydra.main` : This decorator specifies where Hydra should look for the configuration files
  - `version_base=None` is used to avoid warnings about the default version of Hydra.
  - `config_path` : Make sure this path is correct and relative to the location of `get_data.py` . In this case, since `get_data.py` is in `src/ingestion/` , we need to go up two levels (`../../`) to reach the `conf/` directory.
  - `config_name="config"` tells Hydra to load `conf/config.yaml` as the main configuration file, which in turn loads the other YAML files.
- `OmegaConf.to_container` : Now, we access our API parameters through the `cfg` object, which is automatically populated by Hydra based on our YAML files. This allows us to keep our code clean and free of hardcoded values.
- `params['latitude']}N, {params['longitude']` : This is how we access the latitude and longitude values from our configuration when making the API request.
- `openmeteo.weather_api(url, params=params)` : We pass the parameters directly from our configuration to the API client, ensuring that any changes in the YAML files or command line overrides will be reflected in the API call without modifying the code.
- `output_path = os.path.join(cfg.paths.raw_data_dir, filename)` : We also use the path defined in our `conf/paths/default.yaml` to determine where to save the ingested data,

making it easy to change the output directory by simply updating the YAML file.

5. The Power of Command Line Overrides

This is where Hydra truly shines. You no longer need to modify your Python code to change the behaviour of your application.

Action 1: Run with Defaults:

Execute the script normally. It will use the defaults defined in your YAML files (Porto).

In [ ]:
```
# If you're running this in the professor's GitHub repository, make sure to navigate to the
import os
os.chdir("../../")  # Adjust this path if your project is located elsewhere
```

In [ ]:
```
!python src/ingestion/get_data.py
```

Action 2: Override from the Command Line

Imagine we now want to train our model on data from **Faro** (Lat: 37.0187, Lon: -7.9272). We simply pass the new values via the CLI.

In [ ]:
```
!python src/ingestion/get_data.py api.params.latitude=37.0187 api.params.longitude=-7.9272
```

Hydra will intercept these overrides, update the `cfg` object internally, and fetch the data for Faro. By coupling this with DVC (from Lab 2), every time you run this command, DVC can automatically track the newly generated dataset.

6. Hydra's Internal Logging (`outputs/` folder)

You will notice that running the script generated a new folder in your project root called `outputs/`. Hydra automatically creates an isolated directory for every single run, partitioned by date and time. Inside these folders, Hydra saves a copy of the exact `.yaml` configuration used for that specific execution (`.hydra/config.yaml`). This is a powerful fallback mechanism for reproducibility alongside MLflow.

> **Note**: We generally add `outputs/` to our `.gitignore` to avoid bloating the repository, as MLflow handles our primary experiment tracking), so don't forget to add it.

7. Managing MLflow Environments with Hydra

Just as we extracted the API parameters, we must also extract our MLOps infrastructure settings. This allows us to easily switch between a local MLflow server (for testing) and a remote production server without altering the training scripts.

Action 1: Create the MLflow Configuration File

Create a new file at `conf/tracking/mlflow.yaml`:

```
# Configuration for MLflow tracking server
uri: "http://localhost:5000"
experiment_name: "LabX_Hydra_Dynamic_Config"
default_run_name: "Hydra_Managed_Run"
```

Action 2: Update the Root Configuration

Update your `conf/config.yaml` to include this new tracking group:

```yaml
defaults:
  - paths: default
  - api: openmeteo
  - tracking: mlflow  # <-- Added MLflow tracking config
  - _self_

project_name: "EMI-WeatherForecast"
```

Action 3: Create the Integration Script

Now, let us combine everything we have learned: **Strict DVC Lineage** (from Lab 2) combined with **Hydra Dynamic Configuration** (from Lab 3).

Create `src/training/lab3_hydra_integration.py`:

```python
import mlflow
import pandas as pd
import hydra
from omegaconf import DictConfig
from dvc.api import DVCFileSystem
import dvc.api
import os

@hydra.main(version_base=None, config_path="../../conf", config_name="config")
def run_dynamic_experiment(cfg: DictConfig):
    print(f"--- Starting Experiment for {cfg.project_name} ---")

    # 1. Setup MLflow dynamically using Hydra configurations
    mlflow.set_tracking_uri(cfg.tracking.uri)
    mlflow.set_experiment(cfg.tracking.experiment_name)

    # 2. Strict DVC Lineage Tracking (From Lab 2)
    try:
        resource_url = dvc.api.get_url(path=cfg.paths.raw_data_dir, repo='.')
        folder_hash = os.path.basename(resource_url)

        # Connect to the Virtual Filesystem at HEAD
        fs = DVCFileSystem(url=".", rev="HEAD")
        tracked_files = fs.ls(cfg.paths.raw_data_dir)

        csv_files = []
        for f in tracked_files:
            file_path = f["name"] if isinstance(f, dict) else f
            if file_path.endswith(".csv"):
                csv_files.append(file_path)

        if not csv_files:
            print("No CSV files are tracked by DVC yet. Please run 'dvc add'
first.")
            return

        latest_tracked_file = sorted(csv_files)[-1]

        with fs.open(latest_tracked_file) as f:
            df = pd.read_csv(f)

    except Exception as e:
        print(f"Error accessing DVC FileSystem: {e}")
        return

    # 3. Log to MLflow
```

```
    with mlflow.start_run(run_name=cfg.tracking.default_run_name):
        # Log the DVC Hash
        mlflow.log_param("dvc_data_hash", folder_hash)

        # Log Hydra configuration parameters to MLflow for absolute
reproducibility
        mlflow.log_param("api_latitude", cfg.api.params.latitude)
        mlflow.log_param("api_longitude", cfg.api.params.longitude)

        mlflow.log_param("input_file", os.path.basename(latest_tracked_file))
        mlflow.log_metric("dataset_rows", len(df))

        print(f"Successfully logged to MLflow at {cfg.tracking.uri}")
        print(f"Experiment: {cfg.tracking.experiment_name}")
        print(f"DVC Data Hash: {folder_hash}")

if __name__ == "__main__":
    run_dynamic_experiment()
```

Action 4: Test the Dynamic CLI Override

Run the script normally, and it will log to `Lab3_Hydra_Dynamic_Config`. However, if you want to test a new theory without muddying your main experiment dashboard, you can override the MLflow experiment name directly from the terminal:

In [ ]:
```
!python src/training/lab3_hydra_integration.py tracking.experiment_name="Testing_Overrides_D
```

This single command dynamically alters the API fetch location and redirects the MLflow logs to a completely new experiment dashboard.

8. Lab Checklist for Students

1. **YAML Setup:** Do you have `conf/config.yaml`, `conf/api/openmeteo.yaml`, `conf/paths/default.yaml`, and `conf/tracking/mlflow.yaml` correctly structured?
2. **Default Execution:** Did `python src/ingestion/get_data.py` run successfully and save a new CSV in `data/raw/`?
3. **Dynamic Override:** Run the script passing Faro's coordinates (`api.params.latitude=37.0187 api.params.longitude=-7.9272`). Open the resulting CSV file and verify the temperature values are different from your default run.
4. **Dynamic MLflow:** Run `lab3_hydra_integration.py` and verify in the MLflow UI that the new experiment `Lab3_Hydra_Dynamic_Config` was created.
5. **Config Logging:** Check the parameters of that run. Can you see the `api_latitude` and `api_longitude` logged alongside your `dvc_data_hash`?
6. **DVC Sync:** Do not forget to run these commands:
   - `dvc add data/raw` to track the new data with DVC.
   - `dvc commit -q` to commit the changes in DVC.
   - `dvc push` to push the new data to your remote DVC storage (if configured).
   - And of course, `git add .`, `git commit -m "Lab 3 --> Integrate Hydra for dynamic configuration management"`, and `git push` to update your GitHub repository with the new code and configurations.

In [ ]:
```
# Run DVC add and commit the new data
!dvc add data/raw
!dvc commit -q
!dvc push
```